

# Integrate into your CI/CD pipeline

Last updated: Sep. 25, 2025

## About FCS CLI and your CI/CD pipeline

Enable a DevSecOps approach by bringing security scanning directly into your development pipeline with the Falcon Cloud Security command line interface (FCS CLI). By automating infrastructure and container security checks, you can maintain rapid deployment while ensuring vulnerabilities and security concerns are detected and addressed before they reach production.

A well-configured FCS CLI integration can:

- Block deployments of container images that violate image assessment policies
- Prevent IaC templates with misconfigurations from reaching production
- Generate comprehensive security reports for both images and IaC
- Ensure consistent security standards across your development lifecycle

Most teams integrate the FCS CLI into their pipelines to perform two critical security checks: scanning container images during the build process and validating infrastructure as code (IaC) templates before deployment. This automation helps catch security issues early, when they're easiest to fix.

The following options are available for integrating the FCS CLI:

- Our GitHub Action for IaC and image assessment with the FCS CLI, available in the [GitHub Marketplace \[https://github.com/marketplace/actions/crowdstrike-fcs-cli-github-action\]](https://github.com/marketplace/actions/crowdstrike-fcs-cli-github-action)
- Use our [open source GitHub Action implementation \[https://github.com/CrowdStrike/fcs-action\]](https://github.com/CrowdStrike/fcs-action) as a reference to create integrations for other CI/CD platforms.

To maximize the security benefits of your integration, we recommend that you:

- Check the FCS CLI's exit code for any errors
- Include scan results in your build artifacts for auditing and tracking
- Configure appropriate `--timeout` values when scanning large images or complex IaC templates

## Create a custom integration

To use the FCS CLI in an automated process, you might want to download the FCS CLI programmatically. You have three options:

- Download the file compressed binary file. See [Download the FCS CLI programmatically \[/documentation/page/wc4a46fa/integrate-the-fcs-cli-into-your-ci-cd-pipeline#efbb50cd\]](#).
- Download the FCS CLI as a container from within a script or pipeline. See [Download the FCS CLI as a container programmatically \[/documentation/page/wc4a46fa/integrate-the-fcs-cli-into-your-ci-cd-pipeline#scd95c0f\]](#).
- Download the FCS CLI as a container with the Falcon Pull Script to pull the FCS CLI as a container. See [Retrieve Falcon Cloud Security Component Images from the CrowdStrike Registry \[/documentation/page/vc320402/retrieve-falcon-cloud-security-product-images-from-the-crowdstrike-registry\]](#)

## Prerequisites

Before retrieving Falcon Cloud Security images, ensure you have an active CrowdStrike subscription and the required command-line utilities installed in your environment.

The following items are required to retrieve Falcon Cloud Security product images:

- An active CrowdStrike subscription with appropriate entitlements

- Command-line utilities:
  - jq
  - curl (version 7.55.0 or later recommended for security)

## Download the FCS CLI programmatically

To use the FCS CLI in an automated process, you may want to discover the latest version and download the compressed binary file programmatically.

### FCS CLI supported platforms and filenames

The following table lists the supported platforms and related platform name and file name you need for downloading.

| Operating system | Architecture  | Platform name | File name                      |
|------------------|---------------|---------------|--------------------------------|
| macOS            | Apple Silicon | darwin-arm64  | fcs_2.1.0_Darwin_arm64.tar.gz  |
| macOS            | Intel-based   | darwin-amd64  | fcs_2.1.0_Darwin_x86_64.tar.gz |
| Linux            | aarch64       | linux-arm64   | fcs_2.1.0_Linux_arm64.tar.gz   |
| Linux            | x86_64        | linux-amd64   | fcs_2.1.0_Linux_x86_64.tar.gz  |
| Windows          | aarch64       | windows-arm64 | fcs_2.1.0_Windows_arm64.zip    |
| Windows          | x86_64        | windows-amd64 | fcs_2.1.0_Windows_x86_64.zip   |

### Step 1: Create an API client key

Create an API client in the Falcon console to obtain the credentials required for authenticating to CrowdStrike and retrieving security images.

1. In the Falcon console, go to [Support and resources > Resources and tools > API clients and keys \[/api-clients-and-keys\]](#).
2. Click **Add new API client**.
3. Enter a client name and description.
4. Select the API following scopes:
  - Falcon Container CLI : Read / Write
  - Falcon Container Image: Read / Write
  - Infrastructure as Code: Read / Write
5. Click **Add**.
6. Copy the client ID and secret to a secure location.
7. Set environment variables:

```
export FALCON_CLIENT_ID=<YOUR_FALCON_CLIENT_ID>
export FALCON_CLIENT_SECRET=<YOUR_FALCON_CLIENT_SECRET>
export FALCON_API_URL=<YOUR_BASE_URL>
```

Your API base url is provided on the API clients page, but if you missed it there, use this table to find the host name that corresponds with your CrowdStrike cloud.

| Cloud | API base URL                     |
|-------|----------------------------------|
| US-1  | https://api.crowdstrike.com      |
| US-2  | https://api.us-2.crowdstrike.com |

| Cloud    | API base URL                                                                                |
|----------|---------------------------------------------------------------------------------------------|
| EU-1     | <a href="https://api.eu-1.crowdstrike.com">https://api.eu-1.crowdstrike.com</a>             |
| US-GOV-1 | <a href="https://api.laggar.gcw.crowdstrike.com">https://api.laggar.gcw.crowdstrike.com</a> |
| US-GOV-2 | <a href="https://api.us-gov-2.crowdstrike.mil">https://api.us-gov-2.crowdstrike.mil</a>     |

## Step 2: Get an auth token

1. Retrieve an auth token. For info, see [Get an auth token \[documentation/page/db3cc3ee/oauth2-auth-token-apis#zb765564\]](#).

For example:

```
curl --request POST \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "client_id=${FALCON_CLIENT_ID}" \
  --data-urlencode "client_secret=${FALCON_CLIENT_SECRET}" \
  --url "${FALCON_API_URL}/oauth2/token"
```

2. From the response, find the `access_token` and save it.

```
FALCON_ACCESS_TOKEN="aabbcc...112233"
```

Or, when you make the call, you can use `jq` to save the value directly:

```
FALCON_ACCESS_TOKEN=$(curl --request POST \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "client_id=${FALCON_CLIENT_ID}" \
  --data-urlencode "client_secret=${FALCON_CLIENT_SECRET}" \
  --url "${FALCON_API_URL}/oauth2/token" | jq -r '.access_token')
```

## Step 3: Enumerate available versions

Use the [FCS CLI supported platforms and filenames \[documentation/page/wc4a46fa/integrate-the-fcs-cli-into-your-ci-cd-pipeline#c6f4cc6c\]](#) table to find the platform name you require, then request the available versions for that platform.

1. Request a list of available versions, substituting your `platform_name`.

```
FCS_PLATFORM=<platform_name>

curl --request GET \
  --header "Authorization: Bearer ${FALCON_ACCESS_TOKEN}" \
  --url "${FALCON_API_URL}/csdownloads/entities/files/enumerate/v1?
platform=${FCS_PLATFORM}&category=fcs"
```

2. From the output, use the version tag to determine the most recent version.

For example, to get a list of versions of the FCS CLI that work on Apple silicon:

```
FCS_PLATFORM=darwin-arm64
```

If you don't specify a platform name, the list returns the available FCS CLI downloads for all platforms.

## Step 4: Download the file

Use the `file_name` and `version` tags from the previous response to download your file.

1. Request a download link by substituting your chosen filename and version.

```
FCS_FILENAME="<filename>"
FCS_VERSION="<version>"

curl --request GET \
  --header "Authorization: Bearer ${FALCON_ACCESS_TOKEN}" \
  --url "${FALCON_API_URL}/csdownloads/entities/files/download/v1?"
```

```
file_name=${FCS_FILENAME}&file_version=${FCS_VERSION}" | \
jq '.resources | {download_url, file_hash}'
```

2. The jq command result provides your download URL and expected file hash. Use the download URL to get the FCS CLI file.

```
FCS_CLI_LINK="<download_url>"

curl -L -o ${FCS_FILENAME} ${FCS_CLI_LINK}
```

3. After download, compare the file's hash with the expected value.

```
sha256 $FCS_FILENAME
```

## Download the FCS CLI as a container programmatically

In addition to being available as a binary, you can download the FCS CLI as a linux/arm64 container.

Follow these instructions to set up your environment variables, obtain your password to the CrowdStrike registry, and download the FCS CLI container image.

**Note:** You only need to steps 1 to 3 once to obtain the password. You can use the same value repeatedly for authentication with the CrowdStrike registry.

For more information on scanning images with the FCS CLI container image, see [Image scans with the FCS CLI in a container \[documentation/page/wc4a46fa/integrate-the-fcs-cli-into-your-ci-cd-pipeline#x0624e4c\]](#).

### Step 1: Create an API client key

Create an API client in the Falcon console to obtain the credentials required for authenticating to CrowdStrike and retrieving security images.

1. In the Falcon console, go to [Support and resources > Resources and tools > API clients and keys \[api-clients-and-keys\]](#).
2. Click **Add new API client**.
3. Enter a client name and description.
4. Select the API following scopes:
  - Falcon Container CLI : Read / Write
  - Falcon Container Image: Read / Write
  - Infrastructure as Code: Read / Write
5. Click **Add**.
6. Copy the client ID and secret to a secure location.
7. Set environment variables:

```
export FALCON_CLIENT_ID="<YOUR_FALCON_CLIENT_ID>"
export FALCON_CLIENT_SECRET="<YOUR_FALCON_CLIENT_SECRET>"
export FALCON_API_URL="<YOUR_BASE_URL>"
```

Your API base url is provided on the API clients page, but if you missed it there, use this table to find the host name that corresponds with your CrowdStrike cloud.

| Cloud    | API base URL                           |
|----------|----------------------------------------|
| US-1     | https://api.crowdstrike.com            |
| US-2     | https://api.us-2.crowdstrike.com       |
| EU-1     | https://api.eu-1.crowdstrike.com       |
| US-GOV-1 | https://api.laggar.gcw.crowdstrike.com |
| US-GOV-2 | https://api.us-gov-2.crowdstrike.mil   |

## Step 2: Get an auth token

1. Retrieve an auth token. For info, see [Get an auth token \[documentation/page/db3cc3ee/oauth2-auth-token-apis#zb765564\]](#).

For example:

```
curl --request POST \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "client_id=${FALCON_CLIENT_ID}" \
  --data-urlencode "client_secret=${FALCON_CLIENT_SECRET}" \
  --url "${FALCON_API_URL}/oauth2/token"
```

2. From the response, find the access\_token and save it.

```
FALCON_ACCESS_TOKEN="aabbcc...112233"
```

Or, when you make the call, you can use jq to save the value directly:

```
FALCON_ACCESS_TOKEN=$(curl --request POST \
  --header "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "client_id=${FALCON_CLIENT_ID}" \
  --data-urlencode "client_secret=${FALCON_CLIENT_SECRET}" \
  --url "${FALCON_API_URL}/oauth2/token" | jq -r '.access_token')
```

## Step 3: Get your password

Now that you have an auth token to use the APIs, you can request a password for the CrowdStrike registry.

1. Get your authentication password for the CrowdStrike registry, platform\_name.

```
curl --request GET \
  --header "Authorization: Bearer ${FALCON_ACCESS_TOKEN}" \
  --url "${FALCON_API_URL}/iac/entities/image-registry-credentials/v1"
```

2. From the response, find the .resources.token and save it.

```
CS_PASSWORD="aabbcc...112233"
```

Or, when you make the call, you can use jq to save the value directly:

```
CS_PASSWORD=$(curl --request GET \
  --header "Authorization: Bearer ${FALCON_ACCESS_TOKEN}" \
  --url "${FALCON_API_URL}/iac/entities/image-registry-credentials/v1" | jq -r
'.resources.token')
```

## Step 4: Access the CrowdStrike registry

Access the CrowdStrike registry and verify that you can download the FCS CLI as a container image.

Gather needed configuration values from the following table.

| If you log in to this CrowdStrike cloud                                                                     | Use this CrowdStrike registry       | Use this region variable |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------|--------------------------|
| US-1: <a href="https://falcon.crowdstrike.com">https://falcon.crowdstrike.com</a>                           | registry.crowdstrike.com            | us-1                     |
| US-2: <a href="https://falcon.us-2.crowdstrike.com">https://falcon.us-2.crowdstrike.com</a>                 | registry.crowdstrike.com            | us-2                     |
| EU-1: <a href="https://falcon.eu-1.crowdstrike.com">https://falcon.eu-1.crowdstrike.com</a>                 | registry.crowdstrike.com            | eu-1                     |
| US-GOV-1: <a href="https://falcon.laggar.gcw.crowdstrike.com">https://falcon.laggar.gcw.crowdstrike.com</a> | registry.laggar.gcw.crowdstrike.com | us-gov-1                 |
| US-GOV-2: <a href="https://falcon.us-gov-2.crowdstrike.mil">https://falcon.us-gov-2.crowdstrike.mil</a>     | registry.us-gov-2.crowdstrike.mil   | us-gov-2                 |

1. Set an environment variable for your CrowdStrike registry. Use the appropriate registry url from the table.

```
export CS_REGISTRY=<REGISTRY_URL_FROM_THE_TABLE>
```

2. Set an environment variable for the FCS CLI container image. Use the appropriate region variable.

```
export CS_IMAGE=${CS_REGISTRY}/fcs/<REGION_CODE_FROM_THE_TABLE>/release/cs-fcs
```

For example, for US-2, CS\_IMAGE is `registry.crowdstrike.com/fcs/us-2/release/cs-fcs`.

3. Set an environment variable with your registry username. Copy your Customer ID (CID) from

[Host setup and management > Deploy > Sensor downloads \[/hosts/sensor-downloads\]](#) page. Prefix this with `fh` and remove the checksum from the end.

For example, for the CID `0123456789ABCDEFGHIJKLMNQRSTUW-WX`, set the username to `fh-0123456789ABCDEFGHIJKLMNQRSTUW`.

```
export CS_USERNAME=fh-<YOUR_CID>
```

4. Set an environment variable with the FCS CLI container target you want to pull. Generally, our FCS CLI containers are tagged with the same version number as the embedded version of the FCS CLI. To see the latest available containerized version and tag, see

[Latest image versions \[/documentation/page/vc320402/retrieve-falcon-cloud-security-product-images-from-the-crowdstrike-registry#o668380a\]](#).

```
export CS_IMAGE_TAG=<VERSION_TAG>
```

For example, `export CS_IMAGE_TAG=2.1.0`

5. Log in to the registry, using your previously saved password.

```
echo "$CS_PASSWORD" | docker login "$CS_REGISTRY" --username "$CS_USERNAME" --password-stdin
```

6. Download the FCS CLI container target.

```
docker pull ${CS_IMAGE}:${CS_IMAGE_TAG}
```

## Image scans with the FCS CLI in a container

After downloading the FCS CLI tool as a container target, use it with this syntax.

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  $CS_IMAGE:$CS_IMAGE_TAG scan image <IMAGE_NAME> \
  --client-id $FALCON_CLIENT_ID \
  --client-secret $FALCON_CLIENT_SECRET \
  --falcon-region $FALCON_REGION
```

If your host's container runtime socket is not `/var/run/docker.sock`, update the left side of the docker variable directive in the second line `</path/to/your/socket>:/var/run/docker.sock` with the path for your container runtime socket.

**Tip:** If your local user doesn't have adequate permissions to run commands in the container, add `--user root` to your command. We've found this is usually only an issue when working with the containerized FCS CLI from a Mac.

## Sample image scans

For simplicity, these examples assume you have tagged the downloaded FCS CLI container image with the tag `fcs:local`. To run on your system, either use the same tag, or update the tag name in the samples.

### Scan the latest nginx image

The default containerized FCS CLI doesn't have your configuration, so you must pass in your CrowdStrike credentials each time.

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  fcs:local scan image nginx:latest \
  --client-id $FALCON_CLIENT_ID \
  --client-secret $FALCON_CLIENT_SECRET \
  --falcon-region $FALCON_REGION
```

### Scan the latest nginx image faster, for upload only

If you don't need local output, you can make the scan run a bit faster by using `--scan-only` with `--upload`. This combination skips the local output and makes your results available in the Falcon console.

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  fcs:local scan image nginx:latest \
  --scan-only \
  --upload \
  --client-id $FALCON_CLIENT_ID \
  --client-secret $FALCON_CLIENT_SECRET \
  --falcon-region $FALCON_REGION
```

### Scan a local tar file

To scan a tar file on your host, you need to mount the filesystem in the container.

```
docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(pwd)/image.tar:/tmp/image.tar \
fcs:local scan image /tmp/image.tar \
--client-id $FALCON_CLIENT_ID \
--client-secret $FALCON_CLIENT_SECRET \
--falcon-region $FALCON_REGION
```

**Tip:** If your local user doesn't have adequate permissions to run commands in the container, add `--user root` to your command. We've found this is usually only an issue when working with the containerized FCS CLI from a Mac.

## Access image reports from the FCS CLI container

A simple way to access saved image assessment reports from the container is to mount the container filesystem's FCS CLI output folder, `~/ .crowdstrike`.

After running image scans with the added mount directive, image scan reports are located in your hosts filesystem in `./image_assessment/reports`. To save to a different local folder, change the left side of the directive from `$(pwd)` to another path.

```
docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
-v $(pwd):~/ .crowdstrike \
fcs:local scan image <IMAGE_NAME> \
--client-id $FALCON_CLIENT_ID \
--client-secret $FALCON_CLIENT_SECRET \
--falcon-region $FALCON_REGION
```

## Scripting

The FCS CLI is designed with scripting in mind. You can easily build it into your CI/CD pipeline to detect security vulnerabilities, malware, and misconfigurations early in the development process.

For example, in your development environment, you might have a CI/CD pipeline that looks something like this:

- build the target image
- use the FCS CLI to scan the image (whether it's local to the build machine, or in a registry)
- validate whether the image would pass the security requirements and send the developer a go/no-go result for this image
- send the developer a summary of high and critical vulnerabilities and detections

In your production environment, your CI/CD pipeline may look slightly different:

- build the target image
- use the FCS CLI to scan the image
- check if this image should be prevented from running, if so, fail the build
- check if the image requires an alert to be sent, if so, share that alert

To see an example of how we built previous versions of the FCS CLI into a GitHub Action, see [our GitHub project \[https://github.com/CrowdStrike/fcs-action\]](https://github.com/CrowdStrike/fcs-action).

## FCS CLI exit codes

Use the FCS CLI exit codes to determine if the scanned image is safe to run.

- exit code 0: the image met the policy requirements
- exit code 1: the image does not meet the policy requirements and should be blocked
- exit code 2: the image does not meet the policy requirements and an alert should be sent

If there's an error during processing, you'll get one of these exit codes.

- exit code 201: invalid input

- exit code 202: authentication error, check your Falcon client ID, secret, and region
- exit code 203: scan processing error
- exit code 204: error downloading the image assessment report
- exit code 205: error during image assessment report preparation
- exit code 206: error generating SBOM
- exit code 207: scan errors detected, see the report file for detailed error information

The **Scan errors** section of your on-screen or saved report can also inform you of any scan errors and provide additional information that can be useful for troubleshooting the problem.

## Advanced configuration for different pipelines

If you're managing multiple pipelines, such as development vs production, you can use the FCS CLI's configuration profiles to separate information such as the CrowdStrike customer ID (CID) and API keys used when connecting to the CrowdStrike cloud. For more info, see [Profile management \[/documentation/page/c5f61e1f/manage-the-fcs-cli#iea96597\]](#).

◀ Assess images with the FCS CLI[/documentation/page/q8c1b4e7/assess-images-with-the-fcs-cli]